

Spring Boot

- **What is Spring Boot?**
- **Spring Boot** is a framework built on top of the **Spring Framework** that simplifies the development of **Java-based web applications and microservices**.
- It eliminates most of the boilerplate configuration needed for traditional Spring applications.

- **Why Use Spring Boot?**
- **Fast setup:** Create production-ready apps quickly with minimal configuration.
- **Embedded servers:** Comes with **Tomcat**, **Jetty**, or **Undertow** — no need to deploy WAR files manually.
- **Microservices ready:** Ideal for building distributed systems.
- **Auto-configuration:** Spring Boot automatically configures beans based on classpath and properties.
- **Actuator:** Built-in endpoints to monitor and manage apps (health, metrics, etc.).
- **Integration:** Works seamlessly with Spring Data, Security, Cloud, etc.

Core Concepts

Concept	Description
Spring Boot Starter	A set of preconfigured dependencies (e.g., <code>spring-boot-starter-web</code>)
Auto Configuration	Automatically configures beans based on the environment and dependencies
Application Properties / YAML	Configurable values for ports, DB connections, logging, etc.
Spring Boot Actuator	Production-ready features like metrics, health checks
Spring Initializr	Online tool to quickly generate a Spring Boot project

Typical Architecture

A simple Spring Boot application is usually structured like this:

less



```
src/main/java/com/example/demo
```

- |— `controller` → Handles HTTP requests (REST endpoints)
- |— `service` → Business logic
- |— `repository` → Database access (JPA, JDBC)
- |— `DemoApplication.java` → Main class (with `@SpringBootApplication`)

- **Example: Hello World REST API**

- 1. pom.xml dependencies (Maven)**

```
<dependencies>
```

```
  <dependency>
```

```
    <groupId>org.springframework.boot</groupId>
```

```
      <artifactId>spring-boot-starter-  
web</artifactId>
```

```
    </dependency>
```

```
</dependencies>
```

2. Main class

```
import org.springframework.boot.SpringApplication;
import
    org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication

public class DemoApplication {
    public static void main(String[] args) {
        SpringApplication.run(DemoApplication.class, args);
    }
}
```

3. Controller

```
import org.springframework.web.bind.annotation.GetMapping;  
import org.springframework.web.bind.annotation.RestController;
```

```
@RestController  
public class HelloController {  
  
    @GetMapping("/hello")  
    public String sayHello() {  
        return "Hello, Spring Boot!";  
    }  
}
```

Run this app → visit <http://localhost:8080/hello>
you'll see:
Hello, Spring Boot!

Common Spring Boot Starters

Starter	Purpose
<code>spring-boot-starter-web</code>	Build REST APIs or web apps
<code>spring-boot-starter-data-jpa</code>	Work with databases using JPA/Hibernate
<code>spring-boot-starter-security</code>	Add authentication and authorization
<code>spring-boot-starter-test</code>	Unit and integration testing
<code>spring-boot-starter-thymeleaf</code>	Server-side rendering (HTML templates)

- **Next Steps**
- Learn about **Dependency Injection (DI)** and **Inversion of Control (IoC)**.
- Integrate **Spring Data JPA** for databases.
- Add **Spring Security** for authentication.
- Build **microservices** with **Spring Cloud**.

- **What is Flask?**
- **Flask** is a **lightweight and flexible web framework** for **Python**, designed to make it easy to build **web applications, REST APIs, and backend services** quickly.
- It's often called a “**micro-framework**” because it doesn't force you to use specific tools or libraries — you can pick what you need.

- **Why Use Flask?**
- **Lightweight:** Minimal setup and no unnecessary overhead.
- **Flexible:** You can structure your app the way you want.
- **Built-in development server:** Easy to test locally.
- **Jinja2 templates:** For dynamic HTML rendering.
- **REST API friendly:** Perfect for building backend APIs.
- **Extensible:** Add ORM (like SQLAlchemy), authentication, etc., as needed.

Flask vs. Django

Feature	Flask	Django
Type	Micro framework	Full-stack framework
Setup	Minimal, quick	More structured
Flexibility	High	Opinionated
Database Integration	Manual (via SQLAlchemy)	Built-in ORM
Ideal For	APIs, prototypes, small/medium apps	Large, structured projects



Typical Flask App Structure

csharp

my_flask_app/

|

├─ app.py # Main application

├─ static/ # CSS, JS, images

├─ templates/ # HTML templates

└─ requirements.txt # Dependencies

- Example 1: Simple “Hello World” App
- ```
from flask import Flask
```

```
app = Flask(__name__)
```

```
@app.route('/')
def hello():
 return "Hello, Flask!"
```

```
if __name__ == '__main__':
 app.run(debug=True)
```

**Run it:**

```
python app.py
```

Then visit <http://127.0.0.1:5000>

- **Example 2: REST API (JSON Response)**

```
from flask import Flask, jsonify
```

```
app = Flask(__name__)
```

```
@app.route('/api/user')
```

```
def get_user():
```

```
 user = {"name": "Alice", "age": 25}
```

```
 return jsonify(user)
```

```
if __name__ == '__main__':
```

```
 app.run(debug=True)
```

**Output:**

```
{"name": "Alice", "age": 25}
```

## Common Flask Extensions

| Extension          | Purpose                                |
|--------------------|----------------------------------------|
| Flask-SQLAlchemy   | Database ORM integration               |
| Flask-Migrate      | Database migrations                    |
| Flask-Login        | User authentication                    |
| Flask-JWT-Extended | Token-based authentication             |
| Flask-RESTful      | Simplified API structure               |
| Flask-CORS         | Cross-Origin Resource Sharing for APIs |

- **Example 3: Flask + SQLite (Basic CRUD)**

```
from flask import Flask, request, jsonify
from flask_sqlalchemy import SQLAlchemy

app = Flask(__name__)
app.config['SQLALCHEMY_DATABASE_URI'] = 'sqlite:///users.db'
db = SQLAlchemy(app)

class User(db.Model):
 id = db.Column(db.Integer, primary_key=True)
 name = db.Column(db.String(80))
 age = db.Column(db.Integer)

@app.route('/users', methods=['POST'])
def add_user():
 data = request.json
 new_user = User(name=data['name'], age=data['age'])
 db.session.add(new_user)
 db.session.commit()
 return jsonify({"message": "User added!"})

@app.route('/users', methods=['GET'])
def get_users():
 users = User.query.all()
 return jsonify([{"id": u.id, "name": u.name, "age": u.age} for u in users])

if __name__ == '__main__':
 with app.app_context():
 db.create_all()
 app.run(debug=True)
```

## Key Concepts

| Concept                            | Description                                   |
|------------------------------------|-----------------------------------------------|
| Routes ( <code>@app.route</code> ) | Define endpoints (URLs)                       |
| Request & Response                 | Handle HTTP methods (GET, POST, PUT, DELETE)  |
| Templates                          | Dynamic HTML with Jinja2                      |
| Blueprints                         | Organize large apps into modules              |
| Config                             | Manage environment variables and app settings |
| Database ORM                       | Use SQLAlchemy for structured data access     |

## Real-World Uses

Flask is used in:

- REST APIs for web/mobile apps
- Machine learning model serving (e.g., deploy models as APIs)
- Microservices in larger systems
- Prototyping and academic projects